Paper 1: Understanding deep learning requires rethinking generalization

Zhang, Bengio, Hardt, Recht, Vinyals (ICLR 2017)

Bibek Dhungana | CS 6362 Assignment 2 | October 18, 2026

1. Problem statement and summary

1.1 Setup

The training set is $S = \{(x_i, y_i)\}_{i=1}^n$, drawn i.i.d. from an unknown distribution $\mathcal{D}$. Each $x_i \in \mathbb{R}^d$ is an image and $y_i \in \{1, \dots, k\}$ is its class. For CIFAR-10 as used in the paper, $n = 50{,}000$, $k = 10$ and $d = 28 \cdot 28 \cdot 3 = 2352$ after cropping. A network $f_\theta : \mathbb{R}^d \to \mathbb{R}^k$ with weights θ gives one score per class. With a loss ℓ (cross entropy for training, 0–1 loss when reporting errors), the training and test error are

$$\hat{R}_n(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell\big(f_\theta(x_i), y_i\big), \qquad R(\theta) = \mathbb{E}_{(x,y)\sim\mathcal{D}}\, \ell\big(f_\theta(x), y\big).$$

R is estimated on the 10,000 test images, and the generalization gap is $R(\theta) - \hat{R}_n(\theta)$.

The networks in the paper have up to 1.6 million weights for 50,000 images. They reach $\hat{R}_n = 0$ and still have a small test error. The usual explanation bounds the gap by the capacity of the hypothesis class $\mathcal{H}$, for example through the Rademacher complexity

$$\hat{\mathfrak{R}}_n(\mathcal{H}) = \mathbb{E}_\sigma\Big[\sup_{h\in\mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \sigma_i\, h(x_i)\Big],$$

where the $\sigma_i \in \{-1, +1\}$ are independent fair coin flips. With probability at least $1 - \delta$, every $h \in \mathcal{H}$ satisfies $R(h) \le \hat{R}_n(h) + 2\,\hat{\mathfrak{R}}_n(\mathcal{H}) + O\big(\sqrt{\log(1/\delta)/n}\big)$. $\hat{\mathfrak{R}}_n$ measures how well $\mathcal{H}$ can match random labels on the training inputs, and it never looks at the true labels. If $\mathcal{H}$ can fit every labeling, then $\hat{\mathfrak{R}}_n \approx 1$ and the bound only says the gap is at most 2, which is useless for an error between 0 and 1.

1.2 Method: the randomization test

The authors take standard networks with their usual training settings and train them on real CIFAR-10 and on versions where the information has been destroyed:

- *true labels*;
- *partially corrupted labels*: with probability p, a label is replaced by a class drawn uniformly from all k classes;
- *random labels*: the case $p = 1$;
- *shuffled pixels*: one fixed random permutation of the pixels, applied to every train and test image;
- *random pixels*: a different random permutation for each image;
- *Gaussian*: each image replaced by Gaussian noise with the mean and variance of the data.

Nothing else is changed. Figure 1 of the paper shows the result: the small Inception network reaches zero training loss in every case (a), the time to fit grows only moderately with p (b), and the test error goes from 13% to 90%, which is chance, as p goes to 1 (c).

Since the network fits 50,000 random labels, $\hat{\mathfrak{R}}_n(\mathcal{H}) \approx 1$ for this architecture. A bound that depends only on $\mathcal{H}$ gives the same answer for the true-label run (13% test error) and the random-label run (90%), so it cannot explain the first one. The same goes for VC dimension and for uniform stability, which also ignore the labels. Whatever explains generalization has to involve the data and the training algorithm.

1.3 How the rest of the paper uses this

Explicit regularization is not the main reason (Section 3). In Table 1 the Inception network gets 85.75% test accuracy with no regularization and 89.05% with random cropping and weight decay, and it still fits random labels with weight decay on. Regularizers help by a few points, but the network generalizes without them.

A small network can already fit any labels (Theorem 1). A two-layer ReLU network with $2n + d$ weights can represent any function on n points in $\mathbb{R}^d$. Pick $a \in \mathbb{R}^d$ so that the projections $z_i = \langle a, x_i \rangle$ are all distinct, sort them, and place thresholds $b_1 < z_1 < b_2 < z_2 < \cdots < b_n < z_n$. The matrix $A_{ij} = \max(z_i - b_j, 0)$ is lower triangular with a positive diagonal, so it is invertible and $w = A^{-1}y$ fits every target. For CIFAR-10 this is about 102,000 weights, far below 1.6 million. So it is no surprise that the networks *can* represent random labels; the experiment shows that SGD actually *finds* such a solution.

SGD prefers small-norm solutions in the linear case (Section 5). Let $X \in \mathbb{R}^{n \times d}$ stack the inputs and train $f(x) = \langle w, x \rangle$ with SGD from $w_0 = 0$. Each step $w_{t+1} = w_t - \eta_t e_t x_{i_t}$, where e_t is the prediction error on example i_t, adds a multiple of a data point, so $w = X^\top \alpha$ for some $\alpha \in \mathbb{R}^n$. If the solution fits the data, $Xw = y$ gives $XX^\top \alpha = y$, and when $XX^\top$ is invertible ($n \leq d$, independent rows)

$$w = X^\top (XX^\top)^{-1} y.$$

This is the minimum-norm solution of $Xw = y$: any other solution is $w + v$ with $Xv = 0$, and $v \perp w$, so its norm is larger. SGD gets this preference for free from starting at zero. The paper reports 1.2% test error on MNIST with this solution, but it also admits that the norm does not predict test error (wavelet preprocessing raises the norm from 220 to 390 and halves the error), and nothing like it is shown for deep networks.

1.4 Answers to the questions in the assignment

How are random labels generated across epochs, and what would go wrong otherwise? They are drawn once before training and then kept; the paper says the random labels are "fixed and consistent across epochs". If a new label were drawn every time an image is seen, the labels would be fresh noise each epoch. The best prediction would then be the uniform distribution, the loss would stay at $\log 10 \approx 2.30$, and there would be nothing to memorize, so the experiment would say nothing about capacity.

With corruption probability p, what fraction of labels differs from the truth, and what test error should we expect at $p = 1$? A label changes only if it is selected (probability p) and the new class differs from the old one (probability $1 - 1/k$), so the changed fraction is $p(1 - 1/k) = 0.9p$. In my runs the measured fractions were 0.180, 0.357, 0.540, 0.720 and 0.899 for $p = 0.2, \ldots, 1.0$. At $p = 1$ the labels are independent of the images, so any prediction on a test image is right with probability $1/k$ and the expected test error is $1 - 1/k = 0.9$ (the red dashed line in Figure 1c).

What optimizer, learning-rate schedule, preprocessing and regularization do the au-

thors use? From Appendix A: pixels are divided by 255, center-cropped to 28×28, and each image is whitened on its own (subtract its mean, divide by its standard deviation). The optimizer is SGD with momentum 0.9. The initial learning rate is 0.1 for the small Inception and 0.01 for the small AlexNet and the MLPs, and it is multiplied by 0.95 after every epoch. The randomization experiments use no weight decay, dropout or data augmentation. The batch size is not given.

2. Reproduction of Figure 1

2.1 LLM, conversation and code

- **LLM:** Claude (Fable and Opus) for the reproduction. I used Gemini only for background questions, such as what Rademacher complexity is. Toward the end the conversation was so long that the model no longer remembered its early parts.
- **Shared conversation:** `https://chatgpt.com/share/e/6ad56563-6b04-83e9-9b36-5c16f745aded`
- **Code:** `https://github.com/dhunganabibek/reading-and-reproducing-papers/tree/main/paper1:Rethinking-Generalization/code`. The notebook `reproduce_fig1.ipynb` redraws the figure and prints every number in this report from the saved logs.
- I did not give the LLM any existing implementation before the Phase 3 snapshot, which I saved on 10 October 2026.

2.2 Setup and choices

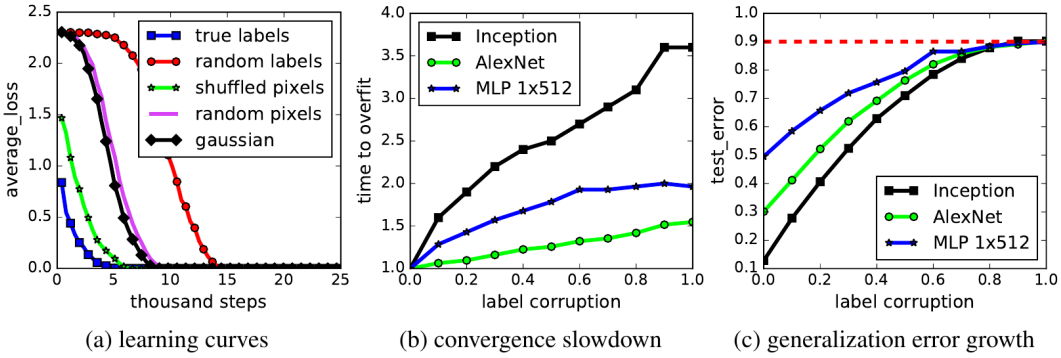
Data	CIFAR-10, 50,000 train / 10,000 test, center crop to 28×28, per-image whitening
Networks	small Inception, small AlexNet, MLP with one hidden layer of 512 units; 1,649,402 / 1,387,786 / 1,209,866 parameters, the same as Table 1 of the paper
Optimizer	SGD, momentum 0.9, learning rate 0.1 (Inception) or 0.01 (others), times 0.95 every epoch; no regularization
Batch size	128; not stated in the paper, but the spacing of the points in its Figure 1a matches (Section 2.6)
Stopping	at 100% training accuracy, or after 100 epochs
Corruption levels	$p = 0$, 0.2, 0.4, 0.6, 0.8, 1.0 for all three networks (the paper uses steps of 0.1)
Random pixels	all 2352 values of an image permuted; a second run keeps the color values of each pixel together
Seeds	one run per setting

Table 1. Setup of my runs.

Convergence time (Figure 1b). The paper plots "time to overfit" relative to the true-label run but never defines it. I evaluate the accuracy on the whole training set after every epoch and count the SGD steps until it first reaches 99%. I could not use 100%: the MLP stops between 99.2% and 99.97% because by then the learning rate has decayed almost to zero. The paper's own Table 1 shows the same thing on random labels (99.34% for this MLP, 99.82% for AlexNet), so its Figure 1b cannot be using 100% either.

Reduced grid. I ran all three architectures from the paper but only 6 of its 11 corruption levels, which kept the Inception sweep to about three GPU hours on my laptop.

2.3 Original and reproduced figure



(a) learning curves (b) convergence slowdown (c) generalization error growth

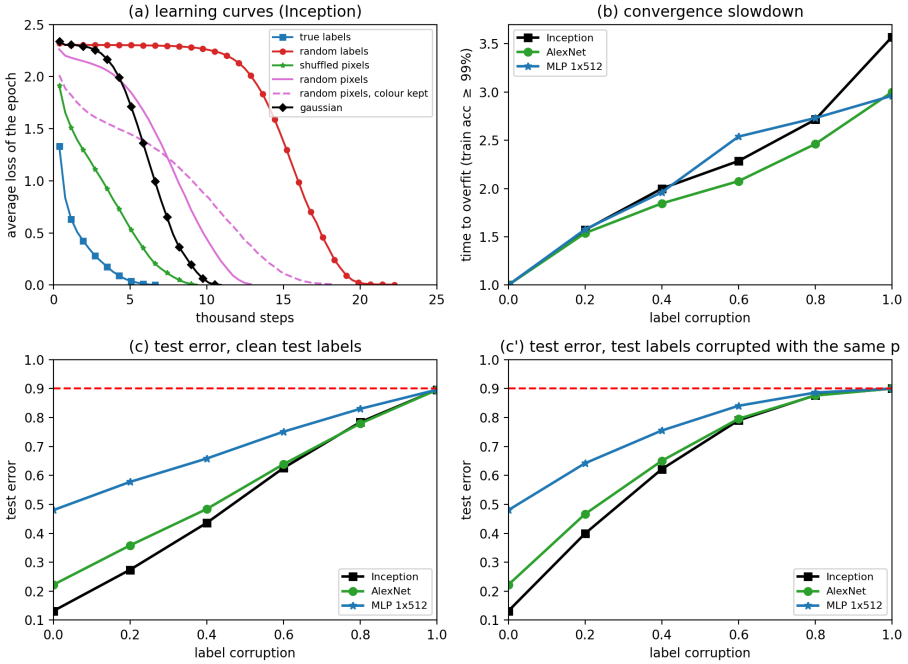


Figure 1. Top: Figure 1 of the paper. Bottom: my reproduction. Panel (c) uses the clean test labels and panel (c′) uses test labels corrupted with the same p as the training labels (Section 2.6).

2.4 Results

(a) Learning curves. All five settings reach zero training loss with the same hyper-parameters, which is the main claim of the paper. The order also matches: true labels reach 100% training accuracy first (6,630 steps), then shuffled pixels (9,360), Gaussian noise (10,920), random pixels (12,870) and random labels (22,230). On random labels the loss stays at $\log 10 = 2.30$ for about 10,000 steps and then drops to zero. I think this is because early on, the gradients from images with unrelated labels point in unrelated directions and mostly cancel; once the network starts to tell individual images apart, memorizing is fast.

(b) Convergence slowdown. More label noise needs more steps, but only a few times more: 3.6 times for Inception at $p = 1$, the same as in the paper.

(c) Test error. For Inception the test error goes from 13% at $p = 0$ to 90% at $p = 1$, while the

training error is zero everywhere. Changing only the labels moves the generalization gap anywhere between 13 and 90 points. Table 2 has all the numbers.

		time to overfit		test error			
						mine, corrupted labels	
network	p	paper	mine	paper	mine, clean	formula	measured
Inception	0.0	1.00	1.00	0.129	0.131	0.131	–
	0.2	1.90	1.57	0.406	0.274	0.399	–
	0.4	2.40	2.00	0.629	0.436	0.622	–
	0.6	2.70	2.29	0.785	0.625	0.790	–
	0.8	3.10	2.71	0.879	0.784	0.877	–
	1.0	3.60	3.57	0.902	0.895	0.900	–
AlexNet	0.0	1.00	1.00	0.301	0.222	0.222	–
	0.2	1.10	1.54	0.522	0.358	0.466	–
	0.4	1.23	1.85	0.692	0.484	0.650	0.654
	0.6	1.32	2.08	0.820	0.639	0.795	0.798
	0.8	1.42	2.46	0.881	0.779	0.876	0.871
	1.0	1.55	3.00	0.901	0.895	0.900	0.900
MLP 1×512	0.0	1.00	1.00	0.495	0.480	0.480	0.480
	0.2	1.43	1.58	0.657	0.577	0.642	0.641
	0.4	1.68	1.96	0.757	0.658	0.755	0.757
	0.6	1.93	2.54	0.865	0.751	0.840	0.840
	0.8	1.96	2.73	0.883	0.830	0.886	0.886
	1.0	1.96	2.96	0.898	0.896	0.900	0.905

Table 2. The paper against my runs. The paper's values are read from the vector graphics of its PDF (`extract_paper_fig1.py`). Time to overfit is relative to $p = 0$. "Corrupted labels" means test labels corrupted with the same p as the training labels: "formula" is computed from the clean accuracy (Section 2.6), "measured" is counted directly, which I only did for the runs made after I added that metric.

2.5 Phases 1–3: failures and corrections

In Phase 1 I gave the LLM the paper PDF and asked it for PyTorch code that reproduces Figure 1. Checking that code against the paper and running it (Phases 2 and 3), I found these problems.

Wrong parameter count. The first small Inception had 1,651,114 parameters, 1,712 more than Table 1 of the paper. That is exactly the number of batch-norm channels: the authors' batch norm has no scale parameter, and their convolutions have no bias. With the scale fixed at 1 the count matched exactly, and so did the counts of the other networks. Matching the AlexNet count (1,387,786) also settled details the text leaves open, such as using two pooling steps down to a 7×7 map.

Convergence time. The first definition was "steps until 100% training accuracy". The MLP never got there, so panel (b) had no MLP curve. I switched to the 99% threshold described above.

Parallel jobs. At first I ran four training jobs on the laptop GPU at once. Two jobs had only been 1.2 times as fast as one, and with four the laptop crashed after 40 minutes. Since then every run goes one at a time and saves a checkpoint after each epoch.

The Inception sweep was still running when I saved the Phase 3 snapshot, so I only noticed the mismatches in the next section after it. The random-pixels and speed checks are my own diagnostics; the explanation for panel (c) came from the authors' demo code in Phase 4.

2.6 What did not match, and why

Test error in panel (c). Against the clean test labels, my test error at intermediate p is well below the paper's: 0.274 against 0.406 at $p = 0.2$ for Inception, and 0.436 against 0.629 at $p = 0.4$. The end points $p = 0$ and $p = 1$ agree. I first checked my own code. The fraction of changed labels is $0.9p$, the preprocessing follows Appendix A, and longer training does not close the gap, because the MLP runs use all 100 epochs and are still below the paper.

The explanation came from the first author's demo, which corrupts the test labels with the same p as the training labels. Let a be the accuracy on clean test labels. A corrupted test label is still the true class with probability $1 - 0.9p$ and is any particular wrong class with probability $p/10$, independently of the network. A prediction counts as correct if it was right and the label was kept, or if it was wrong and the label happened to move onto it:

$$\text{accuracy on corrupted labels} = a\,(1 - 0.9p) + (1 - a)\,\frac{p}{10}.$$

For Inception at $p = 0.4$, $a = 0.564$ gives $0.564 \cdot 0.64 + 0.436 \cdot 0.04 = 0.378$, an error of 0.622, against 0.629 in the paper. Across all p the largest difference for Inception is 0.007 (Table 2). For the MLP and AlexNet I also measured this directly in the training script, and the result agrees with the formula to within 0.005. So the paper's Figure 1c matches corrupted test labels, not clean ones. This matters for how the figure is read: against clean labels the test error grows roughly linearly in p, and the steeper curve in the paper comes from counting the label noise twice, once in training and once in testing.

Random pixels in panel (a). My first version permuted pixel positions but kept the three color values of each pixel together. That keeps the colors of the image, and color alone says something about the class: the loss falls from the start (2.01 in the first epoch, against 2.31 in the paper), fitting takes 18,330 steps, and test accuracy is still 37%. Permuting all 2352 values, channels included, gives a first-epoch loss of 2.26, 12,870 steps and 14% test accuracy. The paper does not say which one it does, but its curve matches the second, so that is the "random pixels" curve in my figure and the first version is the dashed curve.

Speed in panel (a). In absolute steps my Inception is about 1.4 times slower than the paper's. The epoch loss drops below 0.1 after 4,290 steps on true labels (paper: 3,130) and 19,110 steps on random labels (paper: 13,690). The ratios to the true-label run agree, though: random labels take 4.5 times as long in my runs and 4.4 in the paper, Gaussian inputs 2.3 against 2.5, random pixels 2.7 against 2.6. I tested two explanations and both failed. The batch size is not the cause: the points in the paper's Figure 1a are 391 steps apart, exactly one epoch at batch size 128. The initialization is not it either: the authors used TensorFlow, so I reran the true-label run with TensorFlow's default (Glorot uniform), and the first-epoch loss was 1.31 instead of 1.33, while the paper has 0.84. I could not find the cause, and the authors' code is not public.

Slowdown for AlexNet and the MLP in panel (b). The ratio depends on the convergence criterion, which the paper does not give. With my 99% threshold, Inception agrees with the paper (1.6 to 3.6 against 1.9 to 3.6). The MLP agrees at 95% (2.15 at $p = 1$ against 1.96) but not at 99% (2.96). AlexNet does not agree at any threshold: 3.0 against 1.55. The difference is in the true-label run. The paper's AlexNet ratios are multiples of 1/31, so its true-label run took 31 time units and its random-label run 48. If a unit is one epoch, as the spacing in panel (a) suggests, my AlexNet needs a similar 39 epochs on random labels but only 13 on true labels. The paper's AlexNet is also worse at $p = 0$ than in its own Table 1 (30.1% error in the figure, 23.9% in the table; mine has 22.2%). My guess is that the AlexNet in Figure 1 was not trained exactly as Appendix A describes, and I cannot reproduce its curve.

2.7 Phase 4: comparison with the authors' code

The authors did not release the code for the paper. The first author's repository `pluskid/fitting-random-labels` says so and offers a small PyTorch demo with a Wide ResNet and an MLP instead. I compared its protocol with my Phase 3 code. It does the same as mine in three places: labels are corrupted once with a fixed seed, using a uniform draw that can return the true class; the batch size is 128; and training accuracy is measured on the whole training set after each epoch.

The main difference is that the demo **also corrupts the test labels**, which explained panel (c). The demo also uses a step learning-rate schedule and 32×32 inputs; for both I kept the paper's Appendix A. Its 512-unit MLP has 1,577,984 parameters because of the larger input, while the paper's 1,209,866 only works out for a 28×28 crop, which confirms the crop.

Everything I changed after the Phase 3 snapshot:

- a second test metric in `train.py` (accuracy on test labels corrupted with the same p) and panel (c′) in the figure, because of the demo; training itself is unchanged;
- the second random-pixels version and the Glorot initialization option, for my own diagnostics;
- checkpointing and resuming;
- panel (a) drawn with one point per epoch, like the paper;
- the script that reads the paper's plotted values from its PDF.

3. Critical review

3.1 My review

Summary. The paper trains standard image classifiers on labels and inputs that carry no information and shows that they still reach zero training error without changing any settings. From this it argues that capacity measures that ignore the data cannot explain why the same networks generalize on real data. A small expressivity theorem and an analysis of SGD on linear models support the argument.

Strengths. The experiment is simple and cheap, and anyone can check it; I reproduced every qualitative claim of Figure 1 on a laptop. It asks one clear question and answers it. The conclusion is useful mostly as a negative result, because it tells theory where not to look: any explanation has to depend on the data and on the optimizer. The theorem and the linear-model section are modest, and the authors present them that way.

Weaknesses.

- The surprise is partly rhetorical. A model with about 30 weights per training image can memorize the training set, and so can a nearest-neighbor rule. The new fact is narrower: the *same recipe* that generalizes on real data also memorizes noise.
- "Only a small constant factor" undersells the slowdown. It is 3.6 times for Inception, and on random labels the loss does not move for the first 10,000 steps. That plateau shows the optimizer treats noise differently from signal, and the paper does not follow it up.
- Early stopping is dismissed too quickly. With 40% corrupted labels, my Inception had 26% test error after 8 epochs and 44% once it had memorized the training set. The network learns the real pattern first and the noise later, so early stopping does work as a regularizer under label noise. The paper only looks at early stopping on clean data.
- Section 5 does not carry over to deep networks. The minimum-norm result needs a linear model

started at zero, and the section itself shows that the norm does not predict test error.

- There are no error bars. Every curve in Figure 1 is a single run, and so are mine.

Possible errors and unclear points.

- Figure 1c is labeled "test error" but matches test labels that were corrupted like the training labels (Section 2.6). The paper does not say this, and it changes the shape of the curve.
- "Time to overfit" in Figure 1b is never defined. The plotted ratios are multiples of 1/10, 1/31 and 1/28 for the three networks, so it seems to be counted in whole units, probably epochs.
- AlexNet has 30.1% test error at $p = 0$ in Figure 1c but 23.9% in Table 1 (76.07% accuracy without regularization).
- "Random labels" and "random pixels" are not fully specified. For the labels this was only settled in the OpenReview discussion.

3.2 Comparison with the OpenReview reviews

The three reviewers gave 10, 9 and 10, and the paper was accepted as an oral. The program committee called it "groundbreaking" and the evaluation "a model of thoroughness". I number the reviewers as the authors do in their reply. Reviewer 3 valued the paper for raising a good question even without answering it. Reviewer 2 agreed with all the conclusions and only asked the authors to state that the random labels come from the same label set. Reviewer 1 gave a 10 but was the most reserved: most results are "intuitive", and the authors "do not draw a conclusion or offer a strong insight". Reviewer 1 also asked why two networks with the same number of parameters generalize differently.

The public comments were much harsher and closer to my own review. Thomas Dietterich wrote that the results are "completely unsurprising" (replace "neural net" with "decision tree"), that training time "nearly tripled" on random labels, and that early stopping has always been the real control of complexity. Another commenter pointed out that the paper never says which "traditional approaches" predict otherwise, and an anonymous commenter argued that the behavior in Section 5 comes from the zero initialization, not from SGD.

My review shares three points with these comments: the surprise is overstated, the slowdown is not small, and Section 5 does not transfer. I also agree with Reviewer 1 that the paper diagnoses the problem but does not explain it. Two of my points do not appear anywhere in the discussion: the corrupted test labels behind Figure 1c, which I only found by reproducing it, and the measured benefit of early stopping under label noise. Unlike the harshest critics, I think that showing memorization with unchanged hyper-parameters on standard networks, and spelling out what it means for capacity-based bounds, was worth publishing. I would have accepted the paper, but with a score below 10.